

PRÁCTICA 1B

Módulo de registro de votos. Integración de módulos.



de

Raúl Escudero Jiménez

José Luis Pérez Martín

para la asignatura de

ELECTRÓNICA DIGITAL

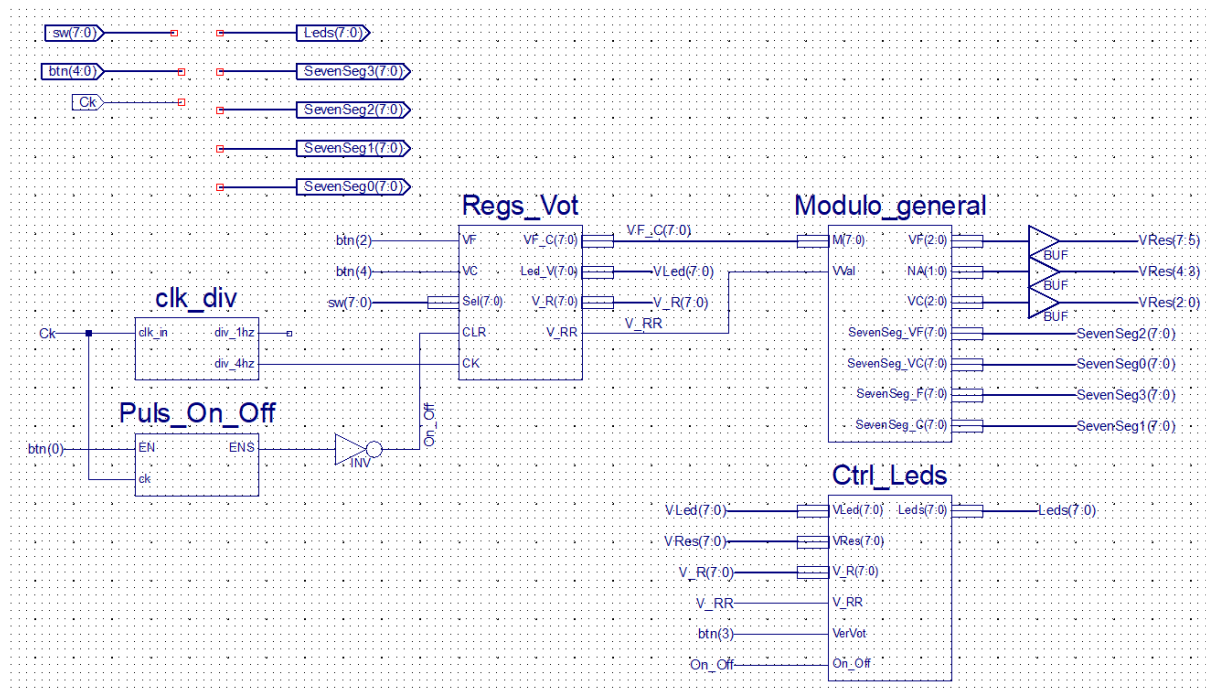
Tutorizado por

Javier López García

1. ESQUEMÁTICOS DEL DISEÑO GLOBAL	5
a) My_Desing	5
b) Reg_Vot	6
c) Regs_Vot	7
d) ENA_GEN	8
e) Puls_On_Off	9
f) Ctrl_Leds	10
2. ANÁLISIS DEL MÓDULO Reg_Vot	11
Estudio del módulo Reg_Vot	11
Tipo de sistema: Moore o Mealy	11
Profundización: MUX 4-1	11
Análisis del MUX_O1	12
Análisis del MUX_O0	12
Profundización FDCs	12
2.2. Tabla de transición de estados	13
Combinación 0 X X	15
Combinación 1 0 0	15
Combinación 1 0 1	16
Combinación 1 1 0	16
Combinación 1 1 1	16
Nota sobre V_R	17
2.3. Diagrama de transición de estados	17
3. CRONOGRAMA DEL MÓDULO Reg_Vot	19
3.1. Código del cronograma del módulo Reg_Vot en lenguaje VHDL	19
3.2. Resultado de simulación (Cronograma) de Reg_Vot	22
3.3. Simulación de Reg_Vot_Exe y Reg_Vot_CASE	24
Código VHDL del módulo Reg_Vot_Exc	24
Código VHDL del Reg_Vot_Exe_TEST	26
Resultado de Simulación del módulo Reg_Vot_Exc	30
Código VHDL del módulo Reg_Vot_CASE	30
Nota sobre Reg_Vot_CASE	33
Código VHDL del Reg_Vot_CASE_TEST	35
Resultado de simulación del módulo Reg_Vot_CASE	38
3.4. Comparación de cronogramas	39
CONCLUSIONES	40

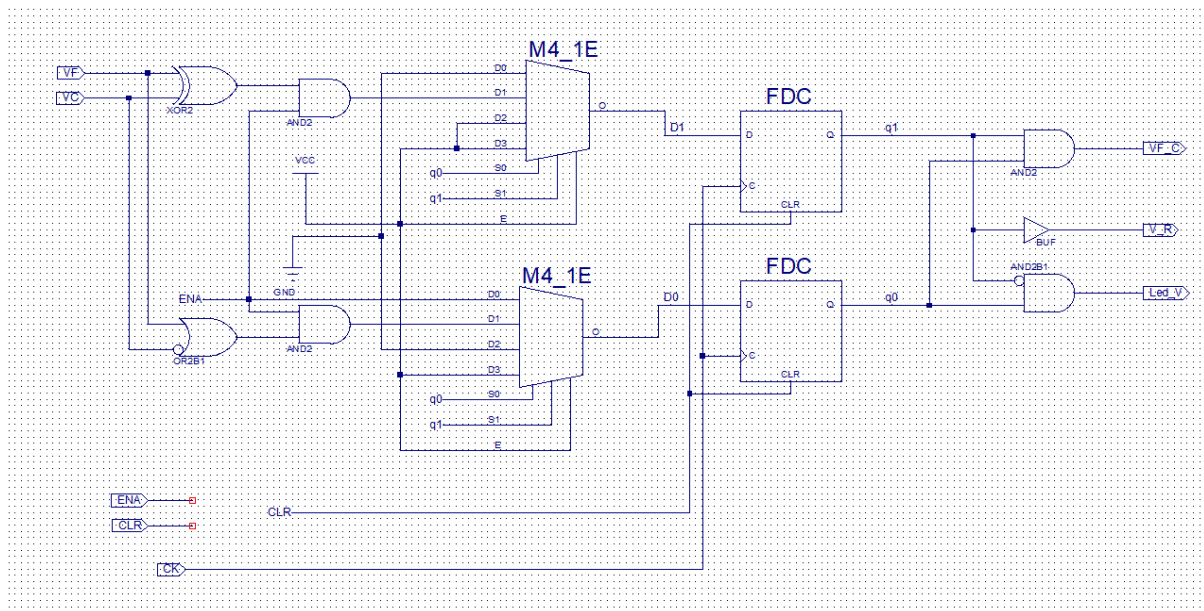
1. ESQUEMÁTICOS DEL DISEÑO GLOBAL

a) My_Desing



Se trata de un bloque que recoge módulos de sistemas tanto combinacionales como secuenciales (Reg_Vot), uniendo todos los módulos. A lo largo de esta sección profundizaremos en cada uno de los módulos que lo componen.

b) Reg_Vot



En cuanto a este módulo, al ser el árbol central de esta práctica, se explicará de forma detallada más adelante en la [sección 2](#), comenzando el análisis mediante las tablas de funcionamiento de tanto los multiplexores como de los flip-flops.

Abajo se muestran las tablas de comportamiento del M4_1 y del FDC —disponibles en la documentación oficial de Xilinx.

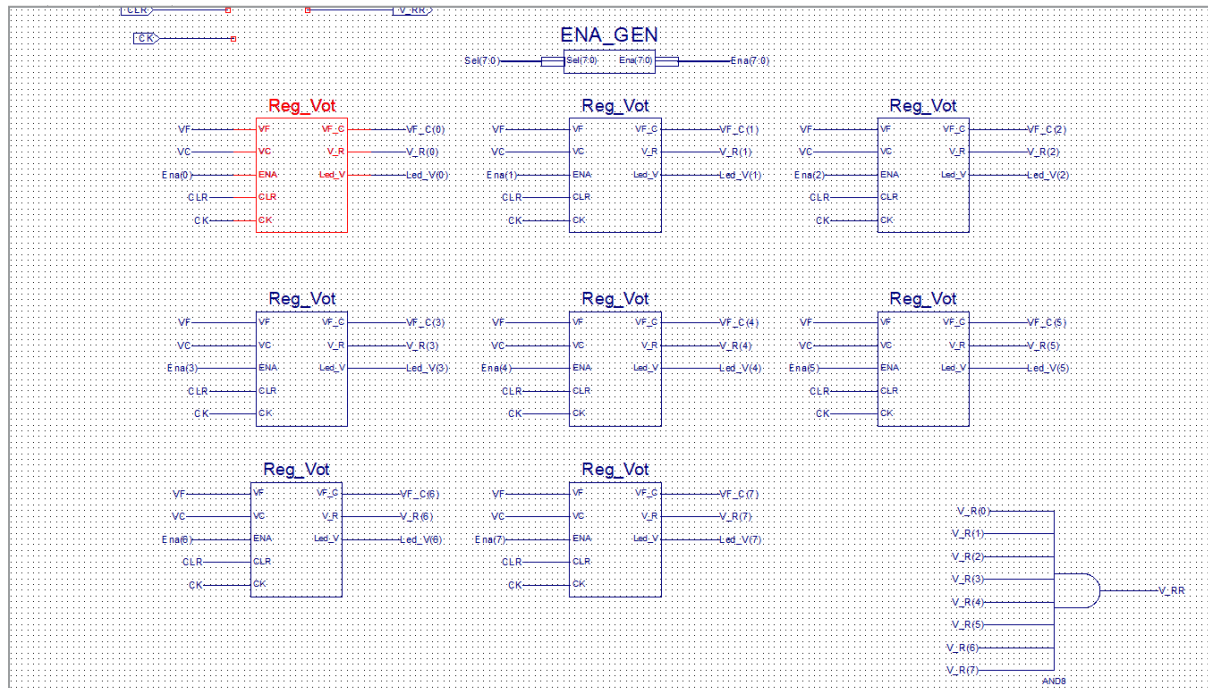
M4_1E: 4-to-1 Multiplexer with Enable

Inputs							Outputs
E	S1	S0	D0	D1	D2	D3	O
0	X	X	X	X	X	X	0
1	0	0	D0	X	X	X	D0
1	0	1	X	D1	X	X	D1
1	1	0	X	X	D2	X	D2
1	1	1	X	X	X	D3	D3

FDC: D Flip-Flop with Asynchronous Clear

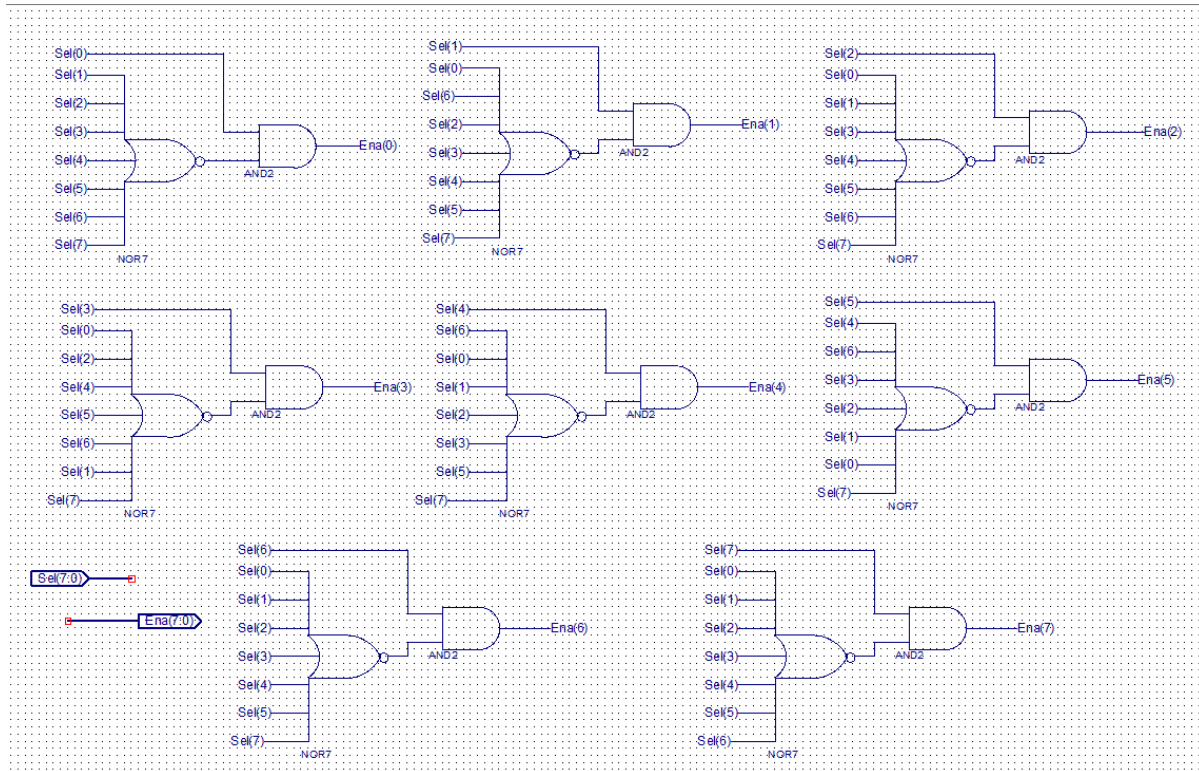
Inputs			Outputs
CLR	D	C	Q
1	X	X	0
0	D	↑	D

c) Regs_Vot



Una vez conocido el funcionamiento de **Reg_Vot**, sabemos que este solo guarda el valor de un voto, luego, para un total de n posibles votantes debe haber un total de n módulos **Reg_Vot**. Este módulo se encarga de almacenar todos los votos del sistema para una misma votación, usando como selector de votante (pues solo puede haber un votante simultáneo) el sub-bloque **ENA_GEN**.

d) ENA_GEN

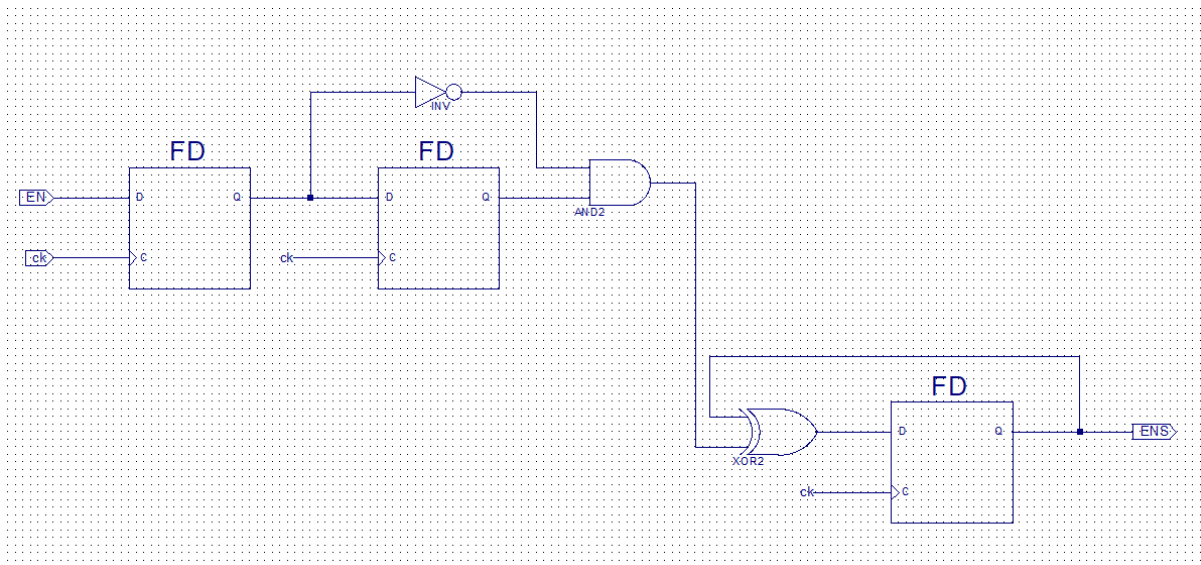


Módulo combinacional cuyo funcionamiento está basado en la correlación o la asociación entre las distintas votaciones guardadas en los Reg_Vot y el módulo que las almacena, el Regs_vot.

$ENA(n) = 1$ si $[Sel(n) = 1]$ AND $[\text{el resto está a 0 (NOR)}]$

Si no está ninguno seleccionado o se seleccionan más de 1 $\Rightarrow$ todos los $ENA(n) = 0$

e) Puls_On_Off



La necesidad de pulsar los botones de manera decidida no se debe a una peculiaridad de la placa Nexys, sino a la ausencia de un sistema de protección frente al rebote mecánico inevitable del pulsador. Este rebote provoca que, durante un intervalo muy corto de tiempo, se generen múltiples transiciones en la señal del botón, que el circuito interpreta como varios pulsos distintos.

Los dos primeros flip-flops, junto con el inversor y la puerta AND, implementan un detector de flanco de bajada: generan un pulso de un solo ciclo de reloj cuando la señal Enable pasa de 1 a 0. El estado Q3 se mantiene constante salvo cuando se detecta ese flanco de bajada, momento en el que Q3 conmuta.

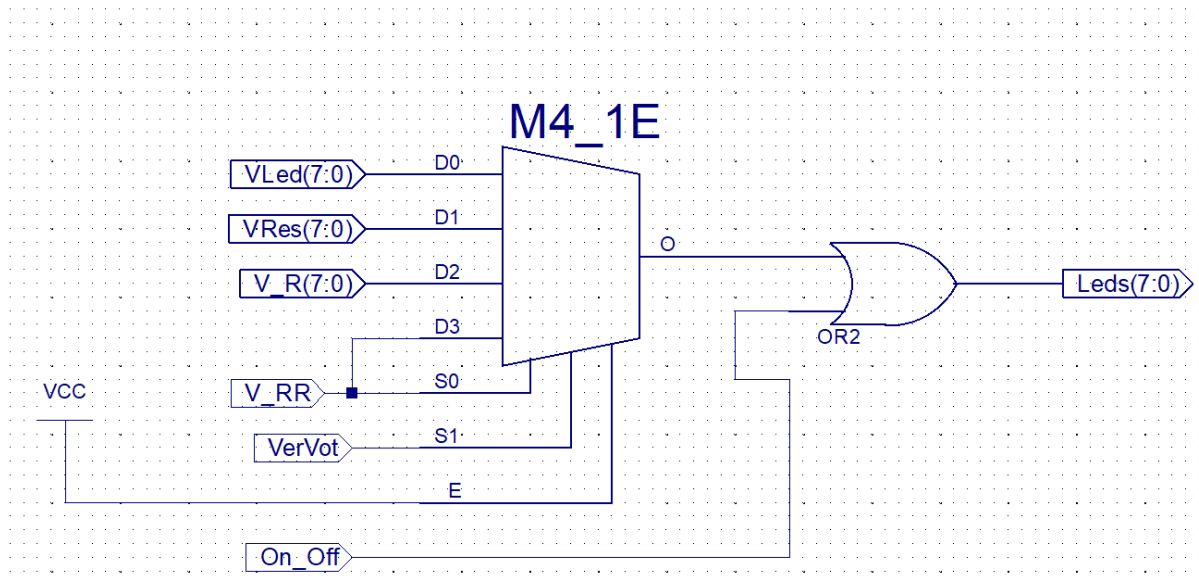
El problema aparece cuando el botón se pulsa o se suelta lentamente: el rebote hace que la señal de Enable experimente varios flancos de subida y bajada muy seguidos. Cada uno de esos falsos flancos puede generar pulsos adicionales en la salida del detector, provocando múltiples conmutaciones de Q3. Como consecuencia, la salida global de este módulo se vuelve impredecible desde el punto de vista del usuario (un solo “clic” puede producir varios cambios de estado).

FD: D Flip-Flop

Inputs		Outputs
D	C	Q
0	↑	0
1	↑	1

$$Q = D$$

f) Ctrl_Leds



Se trata de un sistema combinacional basado en el uso de un multiplexor 4-1 para el control de la tira de leds con la que cuenta la placa Nexys.

El MUX se encuentra siempre activo, pues su entrada de habilitación está conectada a VCC, mientras que la salida total del sistema, **Leds(7:0)**, es controlada mediante una puerta OR cuyas entradas de activación son:

- Al pulsarse el botón 0, dando por encendido el sistema.
- La propia salida del MUX 4-1

Salidas posibles del MUX:

En función de la combinación de entrada del MUX (V_RR y VerVot), la salida del MUX puede tomar 4 casos distintos (D0,D1,D2,D3):

- D0. Enciende el led del votante que ha sido llamado a votar. Cuando un votante es llamado a votación el resto de leds del sistema deben permanecer apagados.
- D1. Corresponde al resultado de la votación, encendiendo los leds(7:5) si la moción fue aceptada, leds (4:3) si la moción no fue aprobada por no tener la mayoría necesaria, y los leds (2:0) si fue rechazada.
- D2. Se encienden los leds de todos aquellos que votaron y se registró su voto en el sistema.
- D3. Correspondiente a si todos los consejeros han tramitado el voto.

2. ANÁLISIS DEL MÓDULO *Reg_Vot*

En esta sección se estudia la composición del módulo *Reg_Vot* previa a la simulación de su comportamiento. Este estudio comprende la explicación del tipo de sistema —Moore o Mealy—, de los submódulos MUX 4-1 y de los Flip-Flops, imprescindibles para poder rellenar la tabla de transición de estados y formular el diagrama de transición de estados.

Estudio del módulo *Reg_Vot*

Sistema secuencial con funcionamiento basado en el registro de voto de un único votante. Como para esta práctica consideramos un total de 8 votantes, el módulo *Regs_Vot* contempla el uso de 8 copias de este mismo módulo, cada una diferenciada para el conjunto de sw(7:0), además de otro módulo denominado *Ena_General*.

Entre el conjunto de puertas lógicas usadas en este circuito multinivel, se destaca el uso de dos multiplexores (MUX 4-1) y dos flip-flops de tipo D (FDC), responsables de la lógica del sistema. Los multiplexores se encargan de seleccionar la salida en función de la combinación de las variables E, q1 y q0, donde q1 y q0 corresponden a las salidas de los FDCs y por tanto componen el estado actual del sistema.

Tipo de sistema: Moore o Mealy

Partiendo del modelo de sistema secuencial de Huffman, tanto el modelo de sistema de Moore como el modelo de Mealy son particularizaciones de este modelo, encontrándonos la única diferencia en la generación de la salida del sistema:

- Para sistemas secuenciales bajo el modelo de Moore, la salida del sistema depende exclusivamente de los valores del estado actual.
- Para sistemas secuenciales bajo el modelo de Mealy, la salida del sistema depende de tanto los valores del estado actual, como de la propia combinación de entrada en ese momento.

Profundización: MUX 4-1

En cuanto a su nomenclatura, vamos a diferenciarlos en función del nombre del cable que conecta su salida con el FDC correspondiente. Para el MUX de arriba, al estar conectado por D1 nombraremos su salida como O1, y para el de abajo de igual forma O0.

En ambos multiplexores la entrada de habilitación está siempre conectada a Vcc luego suprimimos casos con E=0.

Los multiplexores o MUX X-Y son circuitos combinacionales que poseen X entradas de datos, Y salidas y diversas entradas de control.

También la mayoría suele incorporar una entrada de habilitación *Enable*

Su funcionamiento se basa en la generación de la salida bajo la selección de una entrada de datos, la cual fue seleccionada mediante una combinación específica de la entrada de control.

Análisis del MUX_O1

D1 compone la función booleana: $(XOR(VF,VC))*ENA$

Tabla de verdad:

E	q1	q0	O1
1	0	0	D0=GND
1	0	1	D1
1	1	0	D2=1
1	1	1	D3=1

Análisis del MUX_O0

D1 compone una función booleana: $(VF+NOT(VC))*ENABLE$

Tabla de verdad:

E	q1	q0	O1
1	0	0	D0=ENA
1	0	1	D1
1	1	0	D2=0
1	1	1	D3=1

Profundización FDCs

Las celdas de memoria ,y dentro de estas los Flip-flops, componen la unidad básica de funcionamiento de un sistema secuencial, donde la salida del sistema ya no depende exclusivamente de la combinación de entradas (S.Combinacional), depende también del estado del sistema en ese instante. Tenemos constancia de los estados del sistema mediante el uso de celdas básicas de memoria, las cuales, cuentan con la capacidad de almacenar un bit.

Los Flip-flops son elementos de memoria síncronos disparados por flanco de reloj, siendo los utilizados en esta práctica los de tipo D.

Los flip-flops de tipo D tienen una única entrada (además de CK) mediante la cuál se controla la salida o estado siguiente del sistema.

Para el análisis diferenciamos entre FDC1 y FDC0, correspondiente a la salida del MUX. Al haber dos FDCS en el circuito, el número total de estados posibles es 2^2 , es decir 4 estados:

- A: Inicialización del sistema
- B: Llamado de votación
- C: Voto en contra (Btn 4)
- D: Voto a favor (Btn 2)

Tomamos como que ambos Flip-flops se actualizan mediante flanco de subida. La tabla de verdad característica de estos flip-flops D es:

CK	Di	Qi
(flanco up)	0	0
(flanco up)	1	1

Donde Q_i corresponde al estado siguiente y D_i al valor presente en el flip-flop. La salida total del circuito se compone por 3 variables:

- VF_C: tomará valor 0 si el votante optó por el voto en *Contra* y 1 a *Favor*.
- V_R: tomará valor 1 siempre que se termine el proceso de *llamado de votación* y se haya registrado el voto en el sistema.
- Led_V: tomará valor 1 si el sistema se encuentra en la fase de *llamado de votación*, y el valor 0 para cualquiera de los otros 3 estados restantes.

2.2. Tabla de transición de estados

Dividimos la tabla en dos casos debido a la longitud de ésta, siendo el primer caso correspondiente a la entrada de habilitación (ENA) en valor lógico "0" y "1" para el otro caso.

- Entradas: ENA, VF, VC, D1, D0 (correspondientes a los FFD)
- Estados:
 1. Presentes: q1,q0
 2. Futuros: Q1,Q0
- Salidas: VF_C, V_R, Led_V

Al existir 4 estados posibles para una misma entrada, cada entrada será analizada en la tabla un total de 4 veces, dando un total de $4*n$ filas.

n = Combinaciones de entradas posibles con ENA=1

Asignación de estados posibles:

1) Estado inicial «Init»

q1	q0
0	0

Para estos valores de q1 y q0 decimos que el sistema está en fase de arranque, esperando a que el usuario en cuestión sea llamado a votar. En el diagrama aparece etiquetado como «Init».

2) Leds encendidos «Leds_On»

q1	q0
0	1

El estado siguiente del sistema una vez está encendido será solicitar la votación del usuario, manteniéndose en este estado hasta que se pulse:

- a) VF (Btn 2), Voto a favor,
- b) ó VC (Btn 4), Voto en contra.

En el diagrama aparece etiquetado como «Leds_On»

3) Voto registrado en contra «Contra»

q1	q0
1	0

Consideramos esta combinación de q1 y q0 como voto en contra, es decir, se ha pulsado el botón 4 una vez el sistema estaba en la etapa de solicitud de voto. En el diagrama se indica como «Contra».

4) Voto registrado a Favor «Favor»

q1	q0
1	1

Por consiguiente, asignamos esta combinación para el voto a favor, siendo en este caso el botón 2. En el diagrama aparece como «Favor»

Caso ENA=0:

Combinación 0 X X

ENA	VF	VC	q1	q0	Q1	Q0	D1	D0	VF_C	V_R	Led_V
0	X	X	0	0	0	0	0	0	0	0	0
0	X	X	0	1	0	0	0	0	0	0	1
0	X	X	1	0	1	0	1	0	0	1	0
0	X	X	1	1	1	1	1	1	1	1	0

Las “X” (*don’t care*) hacen alusión a la carencia de importancia que tienen las entradas “VF” y “VC” del sistema para la entrada de habilitación con valor “0”. No nos interesan dichos valores puesto que el sistema no permite que estas entradas tengan efecto en el sistema hasta ENA=“1”, por ello, el sistema tiende a mantenerse en el estado actual, guardando el último valor estabilizado.

Caso ENA=1:

Vamos a dividir el análisis de esta etapa en subgrupos de 4, siendo cada grupo de 4 correspondiente a los estados posibles para una misma combinación de entrada:

ENA	VF	VC	q1	q0	Q1	Q0	D1	D0	VF_C	V_R	Led_V
-----	----	----	----	----	----	----	----	----	------	-----	-------

Combinación 1 0 0

1	0	0	0	0	0	1	0	1	0	0	0
1	0	0	0	1	0	1	0	1	0	0	1
1	0	0	1	0	1	0	1	0	0	1	0
1	0	0	1	1	1	1	1	1	1	1	0

Al no pulsar ningún botón, el sistema mantiene su estado, si ya se había registrado un voto; o se dispone en el estado Leds_On, esperando registrar un voto —los flip-flops registran los cambios si se ha pulsado un botón el tiempo suficiente para que el tiempo de establecimiento lo contabilice.

Esta combinación no provoca cambio en el estado del sistema puesto que como condición inicial se toma que la pulsación de ambos botones de forma simultánea no registra el voto, dejando que los estados se mantengan tal y como están.

Nota sobre V_R

El botón 3 Btn 3 —bajo el botón 0— permite visualizar el cómputo total de votos en cualquier momento del proceso. De esta forma, se encenderán los Leds de aquellos votantes cuyo voto haya sido registrado.

2.3. Diagrama de transición de estados

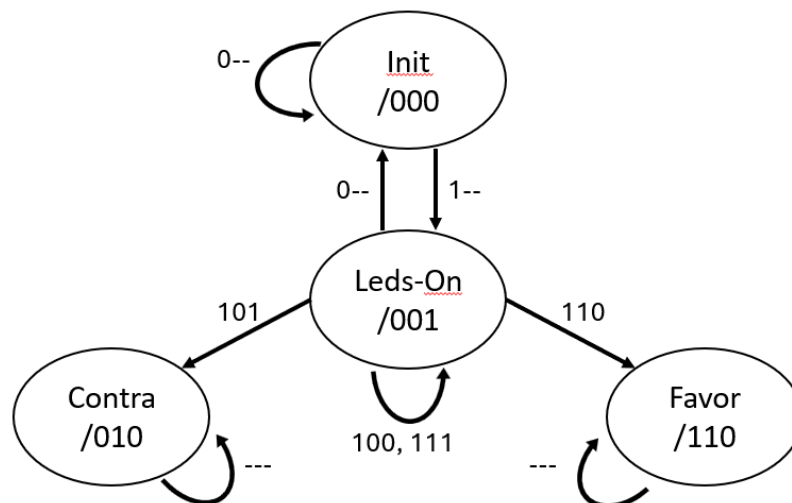
A continuación se muestra el diagrama de transición de estados, que es la representación gráfica de la tabla de transición. En él se indican los estados mediante celdas, junto con la palabra de salida asociada a cada uno.

Dado que el sistema sigue un modelo de Moore, en cada estado se representa una palabra de 3 bits correspondiente a las salidas, en el orden: VF_C, V_R, Led_V.

Las transiciones entre estados se representan mediante flechas etiquetadas con la condición de entrada necesaria para que se produzca el cambio. Cada etiqueta es también una palabra de 3 bits, en el orden: ENA, VF, VC.

Los guiones “—” indican condiciones “no importa” (don’t care): el valor de ese bit de entrada puede ser 0 o 1 sin afectar a la transición; tampoco a las salidas, al tratarse de un sistema de tipo Moore.

Para mayor claridad, se ha omitido el efecto de la entrada de Reset (CLR). Desde cualquier estado, la condición CLR = 1 devuelve al sistema al estado inicial Init de forma asíncrona, y este retorno se mantiene hasta que la señal vuelve a desactivarse.



3. CRONOGRAMA DEL MÓDULO Reg_Vot

Para verificar el funcionamiento del módulo mediante simulación, se genera un cronograma que reproduce la secuencia de eventos propuesta en el guión de prácticas.

3.1. Código del cronograma del módulo *Reg_Vot* en lenguaje VHDL

A diferencia de los test benches de la parte A —puramente combinacionales—, aquí no se recorren todas las combinaciones de entrada como si se tratase de una tabla de verdad.

El objetivo es observar la evolución del estado interno del sistema frente a una secuencia concreta de estímulos. El siguiente código implementa dicha secuencia:

```
--      Vhdl      test      bench      created      from      schematic
C:\Users\Usuario_UMA\Downloads\RLab_Plant_Jose_Raul_07-05-26_1B\Re
g_Vot.sch - Thu May 07 12:27:44 2026
--
-- Notes:
-- 1) This testbench template has been automatically generated
using types
-- std_logic and std_logic_vector for the ports of the unit under
test.
-- Xilinx recommends that these types always be used for the
top-level
-- I/O of a design in order to guarantee that the testbench will
bind
-- correctly to the timing (post-route) simulation model.
-- 2) To use this template as your testbench, change the filename
to any
-- name of your choice with the extension .vhd, and use the
"Source->Add"
-- menu in Project Navigator to import the testbench. Then
-- edit the user defined section below, adding code to generate
the
-- stimulus for your design.
--
```

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;
LIBRARY UNISIM;
USE UNISIM.Vcomponents.ALL;
ENTITY Reg_Vot_Reg_Vot_sch_tb IS
END Reg_Vot_Reg_Vot_sch_tb;

ARCHITECTURE behavioral OF Reg_Vot_Reg_Vot_sch_tb IS
constant CKsemiperiod : TIME := 20 ns;

  COMPONENT Reg_Vot
    PORT( CK          : IN          STD_LOGIC;
          CLR          : IN          STD_LOGIC;
          ENA          : IN          STD_LOGIC;
          VF           : IN          STD_LOGIC;
          VC           : IN          STD_LOGIC;
          VF_C         : OUT         STD_LOGIC;
          V_R          : OUT         STD_LOGIC;
          Led_V        : OUT         STD_LOGIC);
  END COMPONENT;

  SIGNAL CK           : STD_LOGIC;
  SIGNAL CLR          : STD_LOGIC;
  SIGNAL ENA          : STD_LOGIC;
  SIGNAL VF           : STD_LOGIC;
  SIGNAL VC           : STD_LOGIC;
  SIGNAL VF_C         : STD_LOGIC;
  SIGNAL V_R          : STD_LOGIC;
  SIGNAL Led_V        : STD_LOGIC;

BEGIN

  UUT: Reg_Vot PORT MAP(
    VF_C => VF_C,
    V_R  => V_R,
    Led_V => Led_V,
    ENA  => ENA,

```

```

        VC => VC,
        VF => VF,
        CK => CK,
        CLR => CLR
    );

-- *** Test Bench - User Defined Section ***
tb : PROCESS
BEGIN
    CLR <= '0';
    ENA <= '0';
    VF <= '0';
    VC <= '0';

    for i in 0 to 13 loop
        CK <= '1';
        wait for CKsemiperiod;

        case i is
            when 1 =>
                CLR <= '1';
            when 2 =>
                CLR <= '0';
            when 3 =>
                ENA <= '1';
            when 4 =>
                VF <= '1'; VC <= '1';
            when 5 =>
                VC <= '0';
            when 6 =>
                VF <= '0'; VC <= '1';
            when 7 =>
                ENA <= '0';
            when 8 =>
                CLR <= '1';
            when 9 =>
                CLR <= '0'; ENA <= '1'; VF <= '0'; VC <= '0';

```

```

        when 10 =>
            ENA <= '0';
        when 11 =>
            ENA <= '1';
        when 12 =>
            VC <= '1';
        when 13 =>
            ENA <= '0'; VF <= '1';
        when others =>
            null;
    end case;

    CK <= '0';
    wait for CKsemiperiod;
end loop;

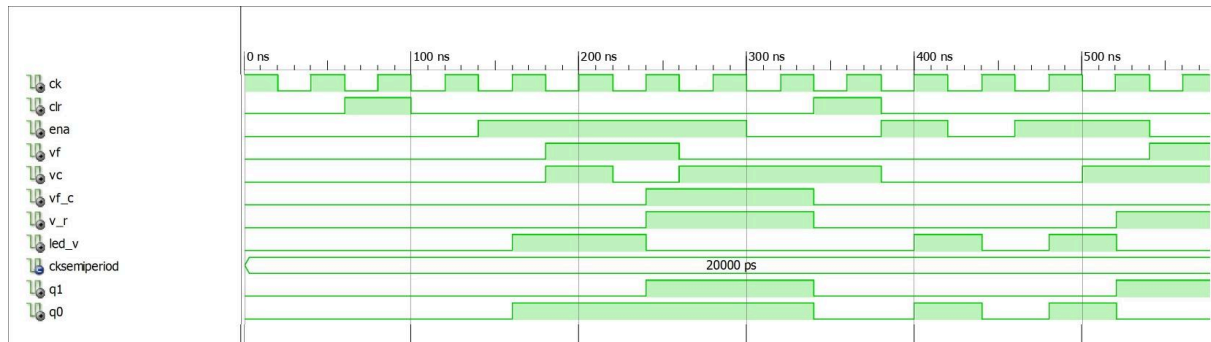
CK <= '1';
wait for CKsemiperiod;
WAIT; -- will wait forever
END PROCESS;
-- *** End Test Bench - User Defined Section ***
END;

```

Tras la inicialización, el proceso recorre los eventos del 1 al 13, aplicando en cada iteración los valores de entrada correspondientes. Aunque el cronograma del guion no especifica completamente la secuencia, esta versión reproduce fielmente el comportamiento esperado del módulo.

3.2. Resultado de simulación (Cronograma) de *Reg_Vot*

Durante la simulación fue necesario localizar en la jerarquía del UUT las señales internas correspondientes al estado del sistema (q0 y q1). Estas señales no son puertos del módulo superior, por lo que no aparecen automáticamente en la ventana de ondas. Se añadieron manualmente desde la jerarquía interna mediante Add to Wave Window, permitiendo observar la evolución temporal del estado interno del registro durante la ejecución del testbench.



La interpretación del cronograma es la siguiente:

1. La simulación comienza con todas las entradas a cero. Se fuerza un pulso de CLR durante un periodo completo de reloj para garantizar el estado inicial.
2. A continuación se activa la señal de habilitación ($ENA = 1$), lo que corresponde al llamado a votación. En el siguiente flanco de subida se enciende $Led_V = 1$, indicando que el votante está habilitado.
3. Con $ENA = 1$, se activan simultáneamente los botones de voto a favor y en contra ($VF = 1$, $VC = 1$). Esta combinación es inválida, por lo que no se registra ningún voto y el sistema permanece en el estado de llamada.
4. Al liberar el botón de voto en contra ($VC = 0$), el siguiente flanco de reloj registra un voto a favor:
 - $V_R = 1$ (voto registrado)
 - $VF_C = 1$ (voto a favor)
 - $Led_V = 0$ (fin de la fase de llamada)
5. Una vez registrado el voto, cualquier cambio posterior en VF o VC no tiene efecto, incluso si se desactiva ENA. El voto permanece almacenado hasta que se active de nuevo CLR.
6. Cuando se pulsa el botón asociado a CLR, el sistema vuelve inmediatamente al estado inicial, de forma asíncrona, sin esperar al flanco de reloj.
7. Finalmente, se comprueba que el sistema responde correctamente a nuevas activaciones de ENA y que registra un voto en contra cuando $VC = 1$ y $VF = 0$.

En paralelo, las señales de estado $q1$ y $q0$ evolucionan correctamente y recorren los cuatro estados posibles de un sistema con dos biestables: $00 \rightarrow 01 \rightarrow 11 \rightarrow 10$, según la secuencia de entradas aplicada y la tabla de transición previamente obtenida.

3.3. Simulación de *Reg_Vot_Exe* y *Reg_Vot_CASE*

En último lugar, se comprueba mediante simulación que ambos módulos programados en lenguaje VHDL, *Reg_Vot_Exe* y *Reg_Vot_CASE*, se comportan de forma análoga al esquemático *Reg_Vot*.

Código VHDL del módulo Reg_Vot_Exc

```

-----
-- Company:
-- Engineer:
--
-- Create Date:      10:02:37 05/07/2026
-- Design Name:
-- Module Name:      Reg_Vot_VHDL_Exc - Behavioral
-- Project Name:
-- Target Devices:
-- Tool versions:
-- Description:
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
-----

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx primitives in this code.
```

```

--library UNISIM;
--use UNISIM.VComponents.all;

entity Reg_Vot_VHDL_Exc is
    Port ( VF : in  STD_LOGIC;
          VC : in  STD_LOGIC;
          ENA : in  STD_LOGIC;
          CLR : in  STD_LOGIC;
          CK : in  STD_LOGIC;
          VF_C : out  STD_LOGIC;
          V_R : out  STD_LOGIC;
          Led_V : out  STD_LOGIC);
end Reg_Vot_VHDL_Exc;

architecture Behavioral of Reg_Vot_VHDL_Exc is
    SIGNAL intVF_C :  STD_LOGIC;
    SIGNAL intV_R :  STD_LOGIC;
    SIGNAL intLed_V :  STD_LOGIC;
    SIGNAL D0
    :
    STD_LOGIC;
    SIGNAL Q0_reg :  STD_LOGIC;
    SIGNAL D1
    :
    STD_LOGIC;
    SIGNAL Q1_reg :  STD_LOGIC;
begin
    D0 <= (NOT(Q0_reg) AND NOT(Q1_reg) AND ENA) OR ((VF OR
not(VC)) AND ENA and
                NOT(Q1_reg) AND Q0_reg) OR (Q0_reg AND Q1_reg
and ENA);
    D1 <= (NOT(Q1_reg) AND Q0_reg AND (VF xor VC) AND ENA) OR
(Q1_reg AND
                NOT(Q0_reg) and ENA) OR (Q0_reg AND Q1_reg
and ENA);
    intVF_C <= Q0_reg AND Q1_reg;
    intV_R <= Q1_reg;

```



```

intLed_V<= not(Q1_reg) AND Q0_reg;

FFD: process (CK, CLR)      -- BIESTABLES
begin
    if CLR = '1' then-- OJO que el clear es activo a
nivel alto
        Q0_reg <= '0';
        Q1_reg <= '0';
    elsif (rising_edge(ck) and ENA = '1') then
        Q0_reg <= D0;    Q1_reg <= D1;
    end if;
end process;

-- Asignación de salidas (lógica combinacional)
VF_C    <= intVF_C;
V_R     <= intV_R;
Led_V <= intLed_V;
end Behavioral;

```

Código VHDL del Reg_Vot_Exe_TEST

```

-----
---
-- Company:
-- Engineer:
--
-- Create Date:    09:54:25 05/14/2026
-- Design Name:
--
--                                Module                                Name:
C:/Users/edie/Desktop/RLab_Plant_Jose_Raul2_07-05-26_1B/Reg_Vot_VHDL_Exc_TEST
.vhd
-- Project Name:  RL_23Pract1B
-- Target Device:

```

```

-- Tool versions:
-- Description:
--
-- VHDL Test Bench Created by ISE for module: Reg_Vot_VHDL_Exc
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
-- Notes:
-- This testbench has been automatically generated using types std_logic and
-- std_logic_vector for the ports of the unit under test.  Xilinx recommends
-- that these types always be used for the top-level I/O of a design in order
-- to guarantee that the testbench will bind correctly to the
post-implementation
-- simulation model.
-----
---
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--USE ieee.numeric_std.ALL;

ENTITY Reg_Vot_VHDL_Exc_TEST IS
END Reg_Vot_VHDL_Exc_TEST;

ARCHITECTURE behavioral OF Reg_Vot_VHDL_Exc_TEST IS
constant CKsemiperiod : TIME := 20 ns;
  COMPONENT Reg_Vot_VHDL_Exc
    PORT( CK      : IN          STD_LOGIC;
          CLR     : IN          STD_LOGIC;
          ENA     : IN          STD_LOGIC;
          VF      : IN          STD_LOGIC;
          VC      : IN          STD_LOGIC;
          VF_C    : OUT         STD_LOGIC;
          V_R     : OUT         STD_LOGIC;
          Led_V   : OUT         STD_LOGIC);
  END COMPONENT;

```

```

END COMPONENT;

SIGNAL CK          :      STD_LOGIC;
SIGNAL CLR          :      STD_LOGIC;
SIGNAL ENA          :      STD_LOGIC;
SIGNAL VF          :      STD_LOGIC;
SIGNAL VC          :      STD_LOGIC;
SIGNAL VF_C        :      STD_LOGIC;
SIGNAL V_R         :      STD_LOGIC;
SIGNAL Led_V       :      STD_LOGIC;

BEGIN

UUT: Reg_Vot_VHDL_Exc PORT MAP (
    VF_C => VF_C,
    V_R => V_R,
    Led_V => Led_V,
    ENA => ENA,
    VC => VC,
    VF => VF,
    CK => CK,
    CLR => CLR
);

-- *** Test Bench - User Defined Section ***
tb : PROCESS
BEGIN
    CLR <= '0';
    ENA <= '0';
    VF <= '0';
    VC <= '0';

    for i in 0 to 13 loop
        CK <= '1';
        wait for CKsemiperiod;

        case i is
            when 1 =>
                CLR <= '1';
            when 2 =>
                CLR <= '0';
        end case;
    end loop;
end tb;

```

```

when 3 =>
    ENA <= '1';
when 4 =>
    VF <= '1'; VC <= '1';
when 5 =>
    VC <= '0';
when 6 =>
    VF <= '0'; VC <= '1';
when 7 =>
    ENA <= '0';
when 8 =>
    CLR <= '1';
when 9 =>
    CLR <= '0'; ENA <= '1'; VF <= '0'; VC <= '0';
when 10 =>
    ENA <= '0';
when 11 =>
    ENA <= '1';
when 12 =>
    VC <= '1';
when 13 =>
    ENA <= '0'; VF <= '1';
when others =>
    null;

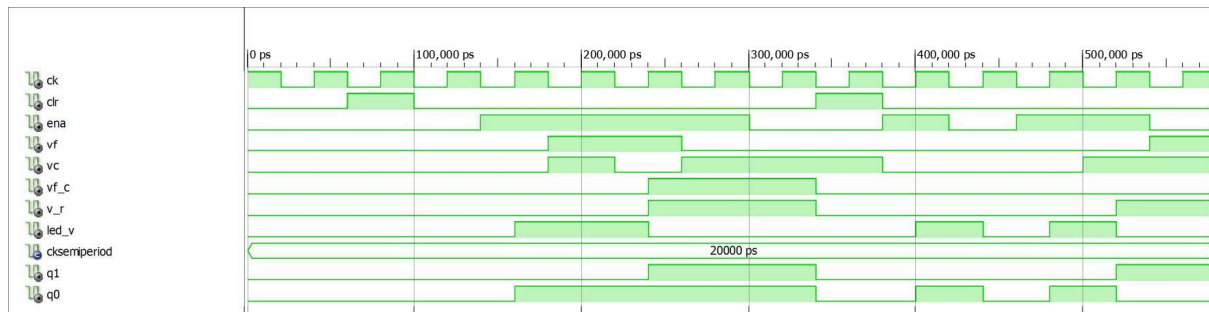
end case;

CK <= '0';
wait for CKsemiperiod;
end loop;

CK <= '1';
wait for CKsemiperiod;
WAIT; -- will wait forever
END PROCESS;
-- *** End Test Bench - User Defined Section ***

END;
```

Resultado de Simulación del módulo Reg_Vot_Exc



Código VHDL del módulo Reg_Vot_CASE

```

-----
--
-- Company:
-- Engineer:
--
-- Create Date:      10:29:15 05/07/2026
-- Design Name:
-- Module Name:      Reg_Vot_VHDL_Case - Behavioral
-- Project Name:
-- Target Devices:
-- Tool versions:
-- Description:
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
-----
-----

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values

```

```

--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx primitives in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity Reg_Vot_VHDL_Case is
    Port ( VF : in  STD_LOGIC;
          VC : in  STD_LOGIC;
          ENA : in  STD_LOGIC;
          CLR : in  STD_LOGIC;
          CK : in  STD_LOGIC;
          VF_C : out  STD_LOGIC;
          V_R : out  STD_LOGIC;
          led_V : out  STD_LOGIC);
end Reg_Vot_VHDL_Case;

architecture Behavioral of Reg_Vot_VHDL_Case is
    signal EST_actual : STD_LOGIC_VECTOR (1 DOWNTO 0);
    signal EST_proximo : STD_LOGIC_VECTOR (1 DOWNTO 0);
begin

    -- registro tipo D
    FFD: process (CK, CLR) -- biestables tipo D
    begin
        if (CLR = '1') then
            EST_actual <= (others => '0');
        else
            if rising_edge(CK) then
                EST_actual <= EST_proximo;
            end if;
        end if;
    end process;

    -- A PARTIR DE AQUI SIGUE LA LÓGICA COMBINACIONAL PARA EL CALCULO
    -- DEL ESTADO SIGUIENTE

```

```
-- AVISO 1: Implementar procesos, no mezclar con instrucciones if,
case, etc.
-- AVISO 2: Si distintas instrucciones asignan valores a una misma
señal
-- puede existir el error de señal afectada por múltiples drivers,
es decir,
-- dos procesos intentan modificar la misma señal...
```

```
process (ENA, VC, VF)
begin
    if (ENA = '1') then
        case EST_actual is
            when "00" => EST_proximo <= "01";
            when "01" => if (VC = '1' and VF = '0') then
EST_proximo <= "10"; end if;
                                if (VF = '1' and VC = '0') then
EST_proximo <= "11"; end if;
                                if (VF = '1' and VC = '1')
then EST_proximo <= "01"; end if;
                                if (VF = '0' and VC = '0') then
EST_proximo <= "01"; end if;
                                when others => EST_proximo <= EST_actual;
            end case;
        else
            if (EST_actual = "11" or EST_actual = "10") then
                EST_proximo <= EST_actual;
            else
                EST_proximo <= "00";
            end if;
        end if;
    end if;
end process;
```

```
-- lógica combinacional de salida
VF_C <= EST_actual(0) and EST_actual(1);
V_R <= EST_actual(1);
led_V <= not(EST_actual(1)) and (EST_actual(0));
```

```

end BEHAVIORAL;

-- synopsys translate_off
configuration CFG_Reg_Vot_VHDL_CASE of Reg_Vot_VHDL_CASE is
    for BEHAVIORAL
        end for;
end CFG_Reg_Vot_VHDL_CASE;
-- synopsys translate_on

```

Nota sobre Reg_Vot_CASE

Se introduce un cambio respecto del código de solución proporcionado en el campus virtual.

En el original:

```

else
    if rising_edge(CK) and ENA = '1' then
        EST_actual <= EST_proximo;
    end if;
end if;
end process;

```

En el nuestro:

```

else
    if rising_edge(CK) then
        EST_actual <= EST_proximo;
    end if;
end if;
end process;

```

El código original describe un Flip Flop tipo D con entrada de habilitación ENA, pero dicha entrada de habilitación no se encuentra presente en el circuito secuencial del esquemático Reg_Vot. El hardware real está compuesto por FFD simples, sin enable.

Se detectó este error en la simulación de Reg_Vot_VHDL_CASE: en la secuencia final, se activa ENA = 1, y los leds se encienden en el siguiente flanco de subida del reloj; no obstante, al desactivarse la señal de habilitación (ENA = 0), los leds permanecieron en encendidos, sin actualizarse.

Esto ocurre porque, en el código original, el estado del sistema solo se actualiza cuando `rising_edge(CK) and ENA = '1'`. Esto no se corresponde con el comportamiento físico del circuito, donde el biestable si se actualiza con cada flanco de reloj.

La solución de esta práctica propone eliminar esta última condición para que el estado se actualice en cada flanco de reloj. La decisión de mantener el estado del sistema se delega a la lógica Case, donde ENA se comporta como una señal combinacional.

Por otro lado, El código original incluía la siguiente asignación:

```
else
    EST_proximo <= "00";
end if;
```

Esto implica que cada vez que Enable se desactiva, el estado siguiente pasa automáticamente a "00", independientemente del estado actual del sistema.

Sin embargo, en el funcionamiento real del módulo, si el voto ya se ha registrado (estados "11" o "10"), el estado debe mantenerse, incluso cuando Enable = 0. Para reflejar este comportamiento, se sustituyó la asignación anterior por:

```
else
    if (EST_actual = "11" or EST_actual = "10") then
        EST_proximo <= EST_actual;
    else
        EST_proximo <= "00";
    end if;
end if;
```

Con este cambio:

- Si el sistema está en un estado de voto registrado ("11" o "10"), el estado permanece inalterado al desactivarse Enable.
- En cualquier otro caso, el estado vuelve a "00" cuando Enable = 0.

Este comportamiento coincide con la especificación funcional: una vez registrado el voto, el sistema no debe borrarlo simplemente porque Enable se desactive.

Código VHDL del Reg_Vot_CASE_TEST

```

-----
-----
-- Company:
-- Engineer:
--
-- Create Date:    10:10:20 05/14/2026
-- Design Name:
--
--                                Module                                Name:
C:/Users/edie/Desktop/RLab_Plant_Jose_Raul2_07-05-26_1B/Reg_Vot_VH
DL_CASE_TEST.vhd
-- Project Name:  RL_23Pract1B
-- Target Device:
-- Tool versions:
-- Description:
--
-- VHDL Test Bench Created by ISE for module: Reg_Vot_VHDL_Case
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
-- Notes:
-- This testbench has been automatically generated using types
std_logic and
-- std_logic_vector for the ports of the unit under test.  Xilinx
recommends
-- that these types always be used for the top-level I/O of a
design in order
-- to guarantee that the testbench will bind correctly to the
post-implementation
-- simulation model.
-----
-----

```

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--USE ieee.numeric_std.ALL;

ENTITY Reg_Vot_VHDL_CASE_TEST IS
END Reg_Vot_VHDL_CASE_TEST;

ARCHITECTURE behavior OF Reg_Vot_VHDL_CASE_TEST IS
    constant CKsemiperiod : TIME := 20 ns;
    COMPONENT Reg_Vot_VHDL_CASE
        PORT( CK          : IN          STD_LOGIC;
              CLR         : IN          STD_LOGIC;
              ENA         : IN          STD_LOGIC;
              VF          : IN          STD_LOGIC;
              VC          : IN          STD_LOGIC;
              VF_C        : OUT STD_LOGIC;
              V_R         : OUT STD_LOGIC;
              Led_V       : OUT STD_LOGIC);
    END COMPONENT;

    SIGNAL CK          : STD_LOGIC;
    SIGNAL CLR         : STD_LOGIC;
    SIGNAL ENA         : STD_LOGIC;
    SIGNAL VF          : STD_LOGIC;
    SIGNAL VC          : STD_LOGIC;
    SIGNAL VF_C        : STD_LOGIC;
    SIGNAL V_R         : STD_LOGIC;
    SIGNAL Led_V       : STD_LOGIC;

BEGIN

    UUT: Reg_Vot_VHDL_CASE PORT MAP (
        VF_C => VF_C,
        V_R => V_R,

```

```

        Led_V => Led_V,
        ENA => ENA,
        VC => VC,
        VF => VF,
        CK => CK,
        CLR => CLR
    );

-- *** Test Bench - User Defined Section ***
tb : PROCESS
BEGIN
    CLR <= '0';
    ENA <= '0';
    VF <= '0';
    VC <= '0';

    for i in 0 to 13 loop
        CK <= '1';
        wait for CKsemiperiod;

        case i is
            when 1 =>
                CLR <= '1';
            when 2 =>
                CLR <= '0';
            when 3 =>
                ENA <= '1';
            when 4 =>
                VF <= '1'; VC <= '1';
            when 5 =>
                VC <= '0';
            when 6 =>
                VF <= '0'; VC <= '1';
            when 7 =>
                ENA <= '0';
            when 8 =>
                CLR <= '1';

```

```

when 9 =>
    CLR <= '0'; ENA <= '1'; VF <= '0'; VC <=
'0';

when 10 =>
    ENA <= '0';

when 11 =>
    ENA <= '1';

when 12 =>
    VC <= '1';

when 13 =>
    ENA <= '0'; VF <= '1';

when others =>
    null;

end case;

CK <= '0';
wait for CKsemiperiod;
end loop;

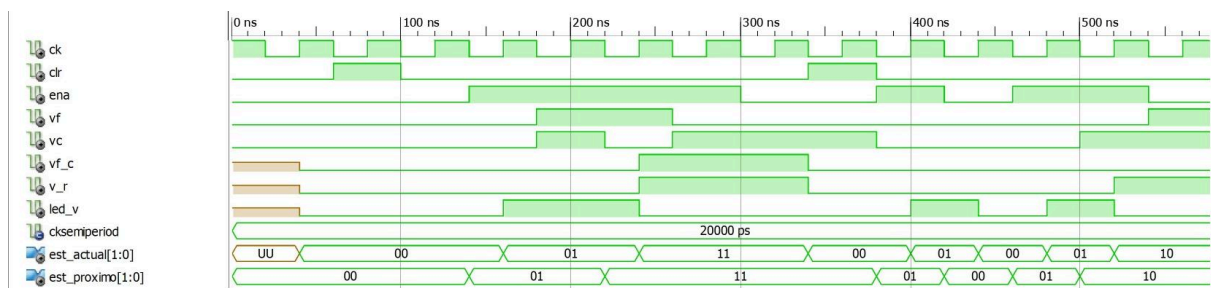
CK <= '1';
wait for CKsemiperiod;
WAIT; -- will wait forever

END PROCESS;

-- *** End Test Bench - User Defined Section ***

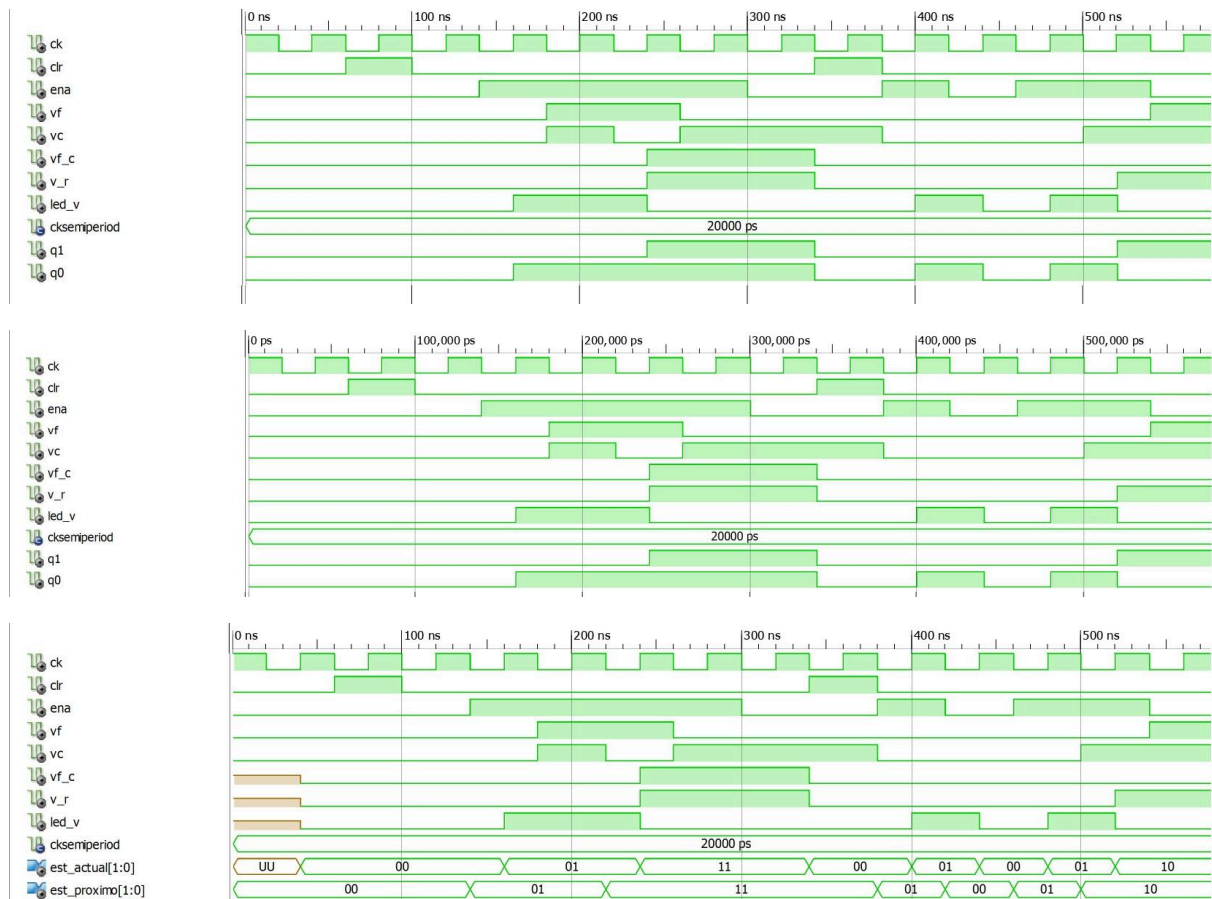
END;
```

Resultado de simulación del módulo Reg_Vot_CASE



Hasta que no se dé el primer ciclo de reloj, el estado y las salidas del sistema permanecen indefinidos. A diferencia de las dos simulaciones anteriores, el estado actual se representa con una palabra de dos bits, correspondiente a q1 y q0.

3.4. Comparación de cronogramas



Se comprueba que los cronogramas de las simulaciones son equivalentes. El de Reg_Vot y Reg_Vot_VHDL_Exc son iguales y el de CASE solo se diferencia en la forma de representar la información, por medio de palabras de 2 bits en lugar de señales digitales.

CONCLUSIONES

En esta práctica se ha analizado en profundidad el funcionamiento del circuito secuencial propuesto. En particular, se estudió el módulo *Reg_Vot*, elaborando su tabla de transición de estados y su diagrama de estados para comprender con precisión su comportamiento interno.

Además del esquemático original, se desarrollaron dos implementaciones en VHDL: *Reg_Vot_Exc* y *Reg_Vot_CASE*. Durante el análisis de este último se detectaron discrepancias respecto al funcionamiento físico del circuito, especialmente en la gestión del estado cuando la señal Enable se desactivaba. Se introdujeron las correcciones necesarias para que la descripción en VHDL reprodujera fielmente el comportamiento del hardware real.

Finalmente, se realizaron simulaciones de los tres módulos y se compararon sus cronogramas. Los resultados muestran que las tres versiones cumplen correctamente las especificaciones del guión de prácticas y presentan un comportamiento temporal equivalente.